Report P1

Group: beta
Authors: Eric Amargant, Eya Ben Said, Marc Bosch Manzano
Professor: Laia Tarrés Benet
Course: Aprenentatge Profund 2025-25268-T1

Summary

1. Introduction 1
2. Methodology 1
3. Results 4
4. Discussion 7
5. Conclusion 8

1. Introduction

Breast cancer is one of the most common cancers worldwide, and early detection greatly improves outcomes. This project uses a Multi-Layer Perceptron to perform binary classification on the breast cancer dataset, exploring how different implementations and optimization methods affect performance. We build the MLP from scratch in NumPy, extend it with momentum, and then reproduce it in PyTorch to compare training behaviour, learning dynamics, and the impact of hyperparameters such as learning rate and hidden layer size.

2. Methodology

Ex1: Multi-Layer Perceptron

In this exercise we have explored and loaded the breast cancer dataset from sci-kit-learn Python library. We have seen the dataset has 569 samples and 30 data dimensions (X) and one target dimension (y), which is 0 if the tumour is malign and 1 if the tumour is benign.

What we have done first in data preparation is to split the dataset into 70% of samples as training set, 15% of validation set, and 15% as testing. We have also taken the first 10 dimensions of input data X.

We have reused the Multi-Layer Perceptron class (MLP) from the example code. The MLP is a kind of neural network used to classify binary target data, so it works fine for our case. In our implementation, we work with one input layer of many dimensions, one hidden layer of many dimensions, and one output layer of one dimension. The class uses sigmoid functions as activation functions, and binary cross-entropy function as loss function. It initializes parameters W, b, weights and biases, respectively, at

small random values for each layer. It also has two methods necessary for learning: forward pass -given *X* data produces prediction *yhat*- and backward pass -given *X* data, *y* target and prediction *yhat* produces the gradients for parameters.

We have also reused the *train* function from the example code. We have modified it in order to compute the validation loss. The *train* function calls the forward and backward passes for many epochs at a given learning rate for the train data to optimize parameters *W* and *b*, and it computes the loss for this train data. Also, it computes the forward pass for validation data, and it computes the loss of this validation data.

In our methodology, we also do plots to check if the data is well learned by the MLP neural network. We comment on those plots in the Results and Discussion sections.

Ex2: Momentum Implementation

This exercise aims to introduce momentum into gradient descent. We implement a new optimizer *OptimMom* based on the provided *OptimSGD*, and use it to train the same model as in Exercise 1. Finally, we compare the training loss obtained for several momentum values in [0, 1].

What is Momentum?

Momentum is a Gradient Descent Optimizer. When we update the weights, we do not only consider the current gradient, but we also take into account the previous direction we were moving in. We keep part of the previous movement and we add a correction from the new gradient. The updates become smoother and less oscillations. Thanks to Momentum, we move faster in the good direction.

Mathematical equations:

$$v_t = \beta v_{t-1} - \eta \nabla J$$
$$W_t = W_{t-1} + v_t$$

- v: velocity
- β: momentum (0-1)
- η: learning rate

Role of beta:

Beta dictates how much memory we want to keep. The larger it is, the more memory we have. Generally, we fix it at 0.9 (we keep 90% of previous velocity and add 10% of the new information).

Momentum initialization:

At the start of the training, there is no previous movement yet. We initialize the velocity term to zero because we don't want bias in any direction.

To answer the exercise: We kept the same NumPy MLP as in the previous exercise since the purpose here was to isolate the effect of the optimizer. The provided *OptimSGD* class was used as a guide. A new class *OptimumMom* was then implemented. Its role was to:

- Store a velocity tensor for each parameter;
- Initialize the velocity to zero;
- Update them at each step using the momentum formula mentioned before, and
- Return the updated parameters.

The training process was as done before. The parameter choices: the experiments were carried out by varying the momentum coefficient in the interval [0,1]. Instead of using a continuous sweep, we used a discrete set of representative values (less calculation time). This makes it easier to observe the effect of increasing momentum without overcrowding the plots. The values chosen were the following: 0, 0.3, 0.6, 0.9. The 0 Momentum is useful because it serves as a baseline (behaves like a gradient descent).

Ex3: Pytorch Implementation

In the last part of the project, the Multi-Layer Perceptron (MLP) developed in Exercise 1 was implemented again using the PyTorch framework. *PyTorch* provides built-in modules for defining neural networks and automatic differentiation (*AutoGrad*), which allows gradients to be computed automatically during training without manually implementing backpropagation.

Before training the *PyTorch* model, the dataset was preprocessed in the same way as in the previous exercises. The labels were converted into PyTorch tensors of type float32 to ensure compatibility with the Binary Cross-Entropy loss function.

To develop the architecture, a custom class named *MLP_PyTorch* was created by inheriting from torch.nn.Module. The architecture consisted of:

- An input layer matching the selected feature subset (10 nodes);
- One fully connected hidden layer (nn.Linear) evaluated at 4, 8, and 16 nodes
- An output layer (nn.Linear) with 1 node for binary classification, and
- Sigmoid activation functions (nn.Sigmoid) applied after both the hidden and output layers.

Stochastic Gradient Descent (*torch.optim.SGD*) was used as the optimizer to update the model parameters.

The hyperparameters were selected to evaluate both convergence stability and model capacity. Three learning rates (0.001, 0.01, 0.05) were chosen to compare slow convergence, stable training, and faster but possible unstable updates. Similarly, hidden layer sizes of 4, 8, and 16 neurons were tested to analyse the effect of increasing model complexity on performance and generalisation. Intuitively, the input and output layer sizes are fixed by the structure of the problem: the input layer must match the number of selected features in the dataset (10), while the output layer is fixed to one neuron because the task is binary classification, where the model predicts a single probability for the positive class. A grid search was conducted over all

combinations of learning rates and hidden neurons, resulting in 9 different model configurations.

3. Results

Ex1:

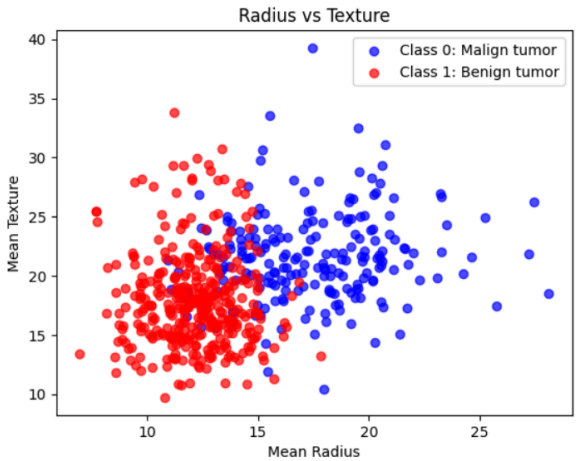

Figure 1.1: Radius and Texture features plot for breast cancer dataset.

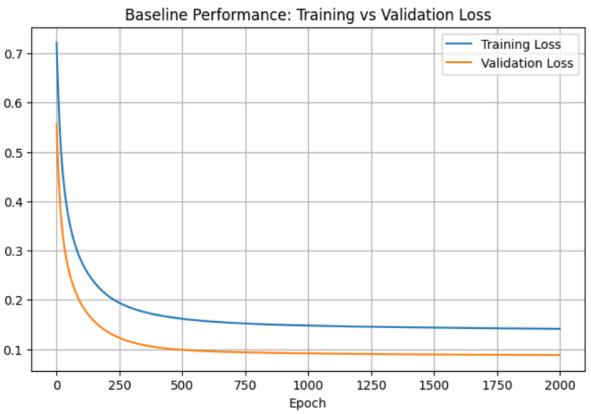

Figure 1.2 Training and Validation Loss for the MLP.

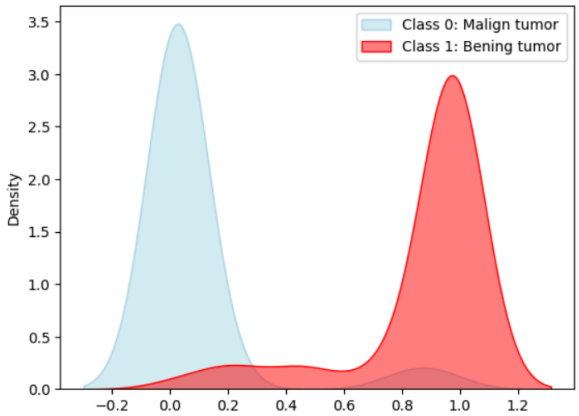

Figure 1.3 Distribution of predicted samples for the Test data.

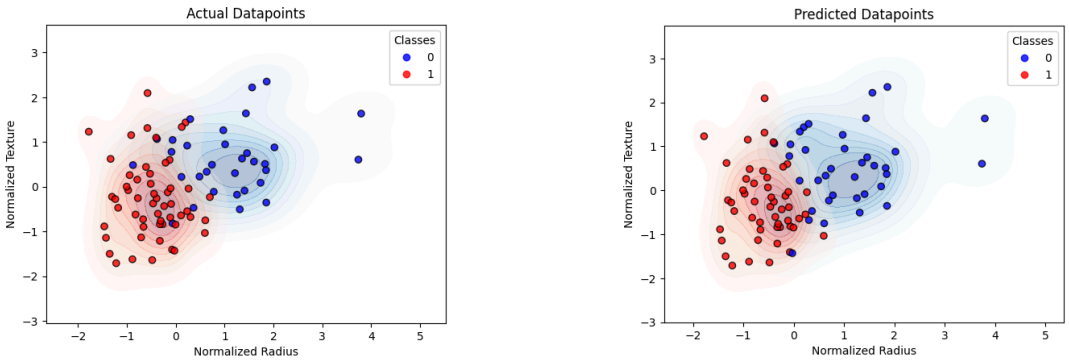

Figure 1.4: Ground-Truth and Predicted Class Distributions of Test Samples in the Radius–Texture Plane

Ex2:

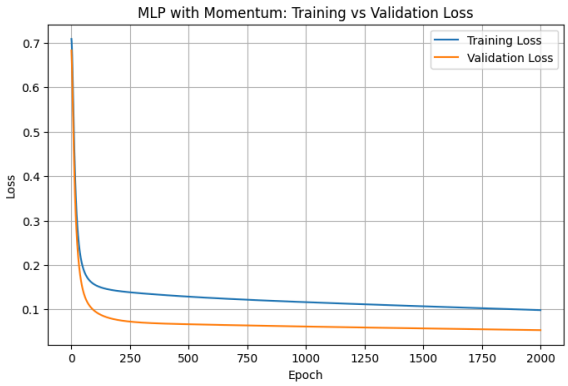

Figure 2.1: Training and Validation Loss for the MLP Trained with Momentum

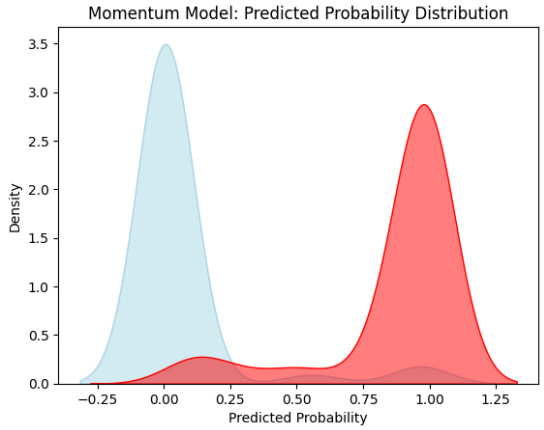

Figure 2.2: Predicted Probability Distribution for the Momentum-Based MLP

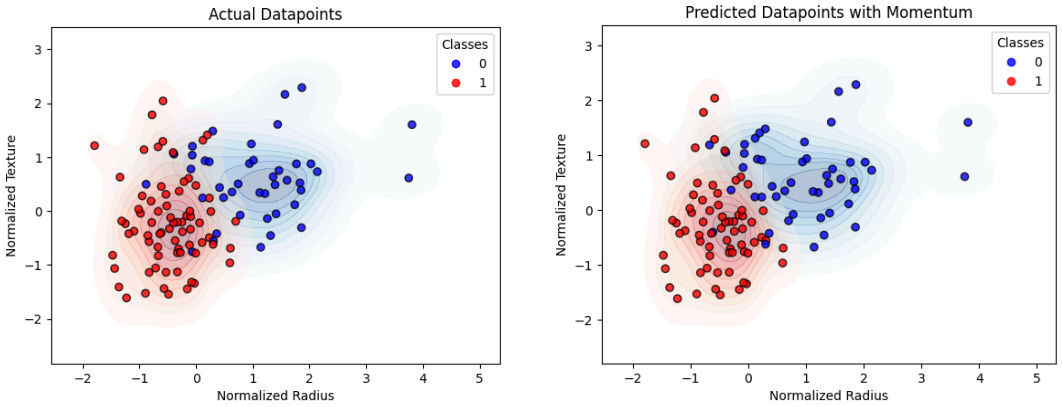

Figure 2.3: Ground-Truth and Predicted Class Distributions of Test Samples in the Radius–Texture Plane

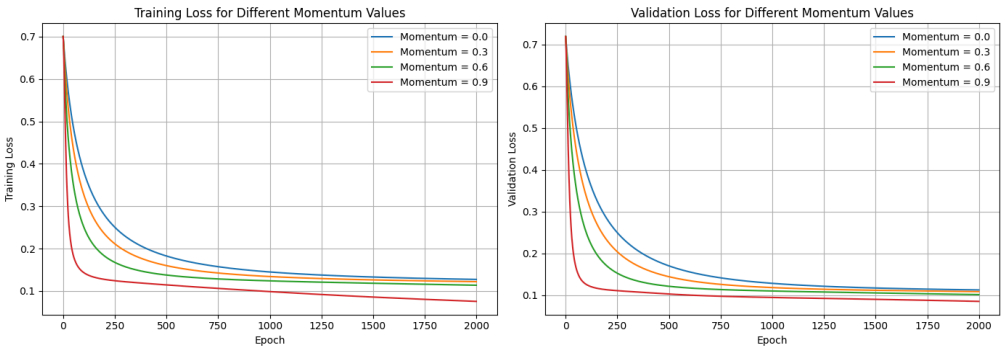

Figure 2.4: Effect of the Momentum Coefficient on Training Loss and Validation Loss.

Momentum Value	Validation Accuracy	Test Accuracy
0.0	0.9670	0.9035
0.3	0.9670	0.8947
0.6	0.9670	0.8947
0.9	0.9670	0.9035

Table 2.1: Classification Accuracy for Different Momentum Coefficients.

Ex3:

Learning Rate	Hidden Neurons	Final Train Loss	Final Val Loss	Validation Accuracy	Test Accuracy
0.01	**16**	0.1477	0.1275	0.9451	0.9386
0.01	**8**	0.1542	0.1338	0.9451	0.9298
0.01	**4**	0.1694	0.1496	0.9451	0.9298
0.05	**16**	0.1232	0.1165	0.9560	0.9035
0.05	**8**	0.1235	0.1164	0.9560	0.9035
0.05	**4**	0.1255	0.1188	0.9560	0.9035
0.001	**16**	0.4364	0.4434	0.8352	0.8860
0.001	**8**	0.5218	0.5256	0.7802	0.8509
0.001	**4**	0.5427	0.5500	0.7363	0.7544

Table 3.1. The following table presents the test analysing nine configurations of learning rate-hidden neurons. The models were evaluated using final training loss, validation loss, validation accuracy, and test accuracy.

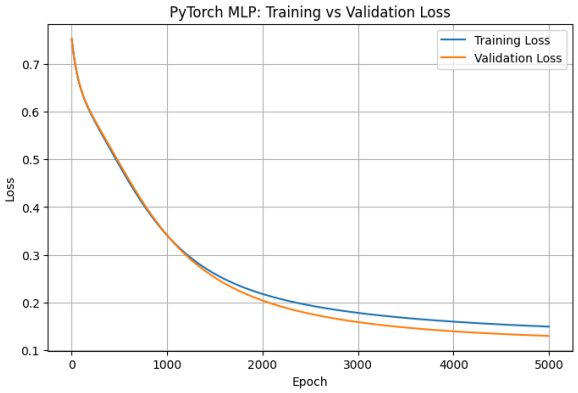

Figure 3.1: Training and validation loss for the initial configuration

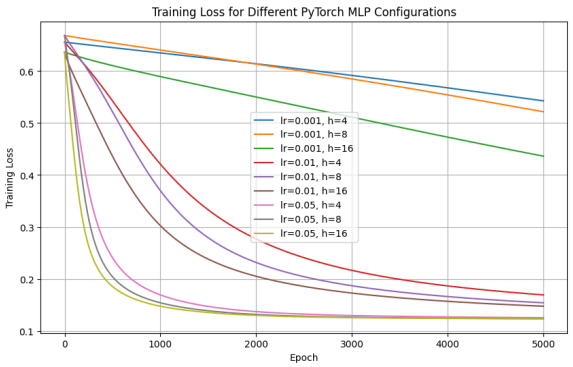

Figure 3.2: Training loss trajectories for all nine grid search configurations

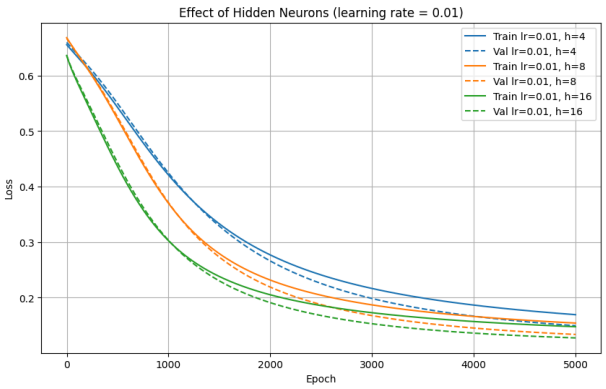

Figure 3.3: Effect of learning rate variation with hidden neurons fixed at 8

Figure 3.4: Effect of hidden neuron variation with learning rate fixed at 0.01

4. Discussion

Here following is a concise discussion about the different exercises mentioning the most important points. For detailed interpretation and discussion of each result, please refer to the code markdowns.

Ex1:

In our two-features plot (Figure 1.1) we can see that the malign samples tend to have higher radius and texture than the benign ones; so this indicates that our data has good behavior to be separated in two classes. The model follows loss curves (Figure 2.2) which tend to be lower, either for training and validation. When doing the prediction with the test data, we have seen that our model has a high accuracy, up to 0.89, and thus, we can consider it classifies well our test samples.

Ex2:

Through analyzing all graphs it becomes apparent that Momentum clearly improves the optimization process: training and validation losses decrease faster and more smoothly (+no visible sign of overfitting).
The predicted probability distributions are well separated, and the predicted class regions closely match the true structure of the data in the radius–texture plane.
The comparison across momentum values shows that higher momentum, especially 0.9, gives the fastest convergence and the lowest losses.
However, Table 2.1 shows that this improvement has only a limited effect on final accuracy, since all momentum values reach the same validation accuracy and very similar test accuracies.
=> Momentum mainly improves convergence speed and training stability rather than the final classification performance.

Ex3:

In Table 3.1., it is observed that moderate learning rates (0.01) achieve the best balance between convergence stability and generalisation performance. Higher learning rates (0.05) converge faster (lower final training and validation losses) but slightly reduce test accuracy, while very small learning rates lead to underfitting due to insufficient convergence within the fixed number of epochs.

Comparison: PyTorch vs. NumPy

Aspect	NumPy	PyTorch
Gradient computation	Manual backpropagation	Automatic differentiation

Implementation effort	More complex and error-prone	Easier to implement and extend
Efficiency	Slower for repeated experiments	Faster and more optimized
Main interest	Understanding how learning works internally	Practical and scalable model development

Limitations and Future Work

Despite the high accuracy achieved, this study presents several limitations. First, the model uses sigmoid activation functions in all layers, which may lead to slower convergence and potential vanishing gradient issues compared to alternatives such as ReLU.
Second, the optimisation process was limited to standard stochastic gradient descent and a simple momentum variant. More advanced optimisers such as Adam or RMSprop could improve convergence speed and stability across different learning rates.
Future work could explore deeper architectures, alternative activation functions, and more advanced optimisation techniques. Additionally, evaluating the model on the full feature set or performing feature selection techniques could provide further insight into which variables contribute most to classification performance.

5. Conclusion

- In this project, a Multi-Layer Perceptron (MLP) was implemented for binary classification on the breast cancer dataset using both NumPy and PyTorch.
- The NumPy implementation helped to understand the full training process, including forward propagation, backpropagation, and parameter updates using gradient descent.
- The effect of using momentum was tested by modifying the optimization process and comparing training behaviour.
- In addition, the same model was implemented using PyTorch, which simplified the training process through automatic differentiation and built-in optimization tools.
- Different learning rates and hidden layer sizes were tested to observe their impact on model performance.
- Overall, the project showed how neural networks can be implemented in different ways and how optimization methods and hyperparameter choices affect training and final results.